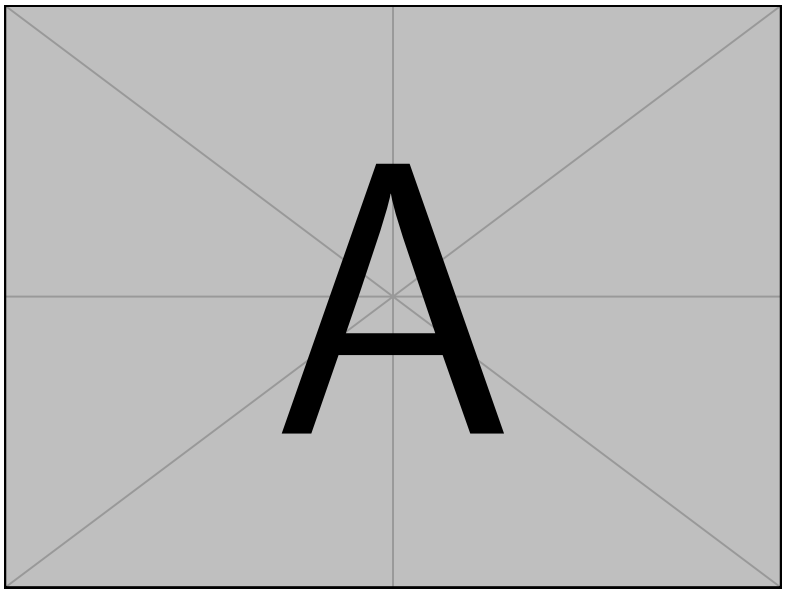
TECHNISCHE UNIVERSITÄT DRESDEN



Schaltkreis- und Systementwurf

Praktikumsbericht

Justus Gottfried Hache (Matrikelnummer)

Studiengang Informationssystemtechnik
Fakultät Elektrotechnik und Informationstechnik
Technische Universität Dresden

14. Oktober 2026

Inhaltsverzeichnis

1. Einleitung **4**

2. Beschreibung des Algorithmus **5**
2.1. Arithmetik 5

3. Entwurf **6**
3.1. Unoptimierter Datenflussgraph 6
3.2. Optimierter Datenflussgraph 6
3.3. Datenpfadarchitektur 6
3.4. Registertransferfolgen 6
3.5. Zustandsgraph 8
3.6. Gesamtschaltung 8

4. Simulation **9**
4.1. Verifikation 9
4.2. Schaltungssynthese 9
4.3. Layoutsynthese 9

5. Zusammenfassung **10**

A. Quellcode **11**
A.1. Referenzimplementierung 11
A.2. FSM 11
A.3. Controllogic 11

B. Reports **12**
B.1. Area 12
B.2. Power 12
B.3. Timing 12

1. Einleitung

Diese Arbeit dokumentiert das Design eines double-percision FFT-ASICs er wurde für das Modul „Schaltkreis und Systementwurf“ entwickelt. Die Aufgabenstellung umfasst dabei die Analyse des gewählten Algorithmus, eine sequentielle Referenzimplementierung, planung der Implementierung in Hardware mithilfe von Registertransferfolgen, sowie die tatsächliche Implementierung mit dem Cadence Design Framework 2. Im finalen schritt wird das Design syntetisiert, simuliert und verifiziert.

Die Fourier Transformation ist eine der wichtigsten Funktionen in der heutigen Zeit. Sie wird für Komprimierungsalgorithmen, Audio und Bildbearbeitung, Analyse von Signalen, Übertragung von Signalen und noch vielen anderen Bereichen verwendet. Die zentrale Eigenschaft der Fourier Transformation ist dabei, aus einer zeitlichen Folge die Amplituden der Frequenzkomponenten zu berechnen. Lange Zeit war es nur möglich, die diskrete Version der Fourier Transformation mit einer Laufzeit von $O = N^2$ zu berechnen. Erst in den 60er Jahren änderte sich dies mit einem der Schnellen Fourier Transformation, beschrieben in Die Zentrale Erkenntnis dabei war, dass man sich durch die periodische Natur der Exponentialfunktion viele Rechnungen sparen konnte. In ihrer Arbeit beschreiben Colley und Turkey, wie durch umformen der gleichungen die Fourier Transformation so geschrieben werden kann, dass nun eine Laufzeit von $O = N \log_N$erreicht werden kann. Dies ermöglicht es auch lange Folgen mit hoher Frequenzauflösung zu berechnen und heutzutage z.B. die Verwendung in Komprimierungsalgorithmen und die Fähigkeit in Echtzeit auf eingebetteten systemen zu laufen.Diese Arbeit wird sich damit befassen, wie man einen AISC (Application Specific Integrated Circuit) erstellen kann, welcher diese Berechnung noch schneller durchführen kann, als es ein Prozessor in gleicher Technologie könnte.

Zitiert wird so: [**munafo_escape_2023**]. Die `.bib` Datei wurde mit Zotero erzeugt. Bitte beim Export auf das neue Format bib**la**tex achten.

2. Beschreibung des Algorithmus

Wie in der Einleitung beschrieben, berechnet die FFT effizient aus einer Zeitlichen Folge an Werten die Frequenzkomponenten. Die finale Implementierung ist außerdem darauf ausgelegt, dass die Berechnung in-place stattfindet, also kein Zusätzlicher Speicher als der Eingabespeicher, welcher gleichzeitig Ausgabespeicher ist, benötigt wird.

Eingabe Der Algorithmus benötigt als Eingabe eine Liste an Werten, welche in gleichmäßigen Abständen aufgenommen wurden und als weitere Eingabe muss die länge der Liste angegeben werden. Die Länge der Liste kann dabei nur eine Potenz von 2 sein. Sollte die Länge der Zahlenfolge kürzer sein, so muss der restliche Speicher mit Nullen aufgefüllt werden.

Ausgabe Der Algorithmus liefert eine Liste an Werten, welche den Amplituden der Frequenzanteile entspricht. Diese Liste hat dabei die gleiche Länge, wie die Eingabe.

Implementierung Ein einfacher weg, die FFT zu veranschaulichen ist der rekursive ansatz. Dieser funktioniert, indem die Originale Zahlenfolge in 2 Teile zerlegt wird, ein Teil mit den Graden Indizes und ein Teil mit den ungeraden. Für jeden dieser Teile wird nun selbst eine FFT berechnet, die Ergebnisse werden schließlich miteinander verrechnet. Wenn die beiden vorherigen Ergebnisse $E[k]$ und $O[k]$ sind, gilt dann für das Gesamtergebnis X mit der Länge N:

$$\begin{aligned} X[k] &= E[k] + W_N^k O[k] \\ X[k + N/2] &= E[k] - W_N^k O[k] \\ k &= 0, \ldots, N/2 - 1 \end{aligned} \tag{1}$$

Man benötigt also für die Kombination N komplexe Multiplikationen und Additionen. Durch wiederholtes Teilen zerlegt man die Zahlenfolge nun solange bis nur noch ein Element übrig ist. Dies wird erreicht, nach $\log_2 N$ Teilen. Interessanterweise ist die Teilung in 2 Teile nicht der Effizienteste Weg, wenn man nur auf die Komplexen Multiplikationen schaut. Laut dem originalem Paper TODO: reference ist eine Teilung in 3 Teile 6% effizienter, eine Teilung in 4 gleich schnell im Vergleich zu einer Teilung in 2. Allerdings Bringt die Teilung in 2 deutlich vereinfachungen der Speicherung und Adressberechnung auf binärkomputern, sodass heutzutage meist eine Teilung in 2, einige praktische implementierungen TODO:reference aber auch eine Teilung in 4 für eine bessere Hardwarenutzung verwenden. Die Referenzimplementierung TODO: link verwendet eine sequentielle Version dieses Algorithmus. Dabei wird jedes Mal Teilen einer neuen „Stufe“ zugeordnet. Die erste Stufe kombiniert dann N Teile, zu $N/2$ die letzte Stufe kombiniert lediglich 2. Dadurch kann eine feste Schleife erstellt werden, welche den Anfangsindex der jeweiligen in einer Stufe berechnet und in einer zweiten schleife wird dann durch die Elemente eines Teils gegangen.

2.1. Arithmetik

Was zum Rundungsverhalten/Abweichungen erklären, usw.

3. Entwurf

3.1. Unoptimierter Datenflussgraph

3.2. Optimierter Datenflussgraph

Auch erklären, was das Optimierungskriterium ist.

3.3. Datenpfadarchitektur

In Tabelle 1 sind die Datentransferoperationen für die Register, Recheneinheiten und den Speicher aufgelistet. Sie wurden aus dem Datenflussgraphen abgeleitet. Zeigt mehr als eine Quelle auf ein Datenpfadelement, wird ein Multiplexer benötigt.

Ziel	Quellen	1	2	3	4	5	6	7	8
R1	←	0	T1						
R3	←	0	T1						
RR	←	0	T1						
RC	←	0	T1						
R2	←	0	T2						
R4	←	0	T2						
RCX	←	T1							
RCY	←	T1							
RD	←	T3							
Rres	←	MEM(DO)							
Riter	←	MEM(DO)							
RI	←	Riter	T4						
Radr	←	T4							
MEM(ADR)	←	Radr							
MEM(DI)	←	RI							
ALU(1)	←	T1	T2	2	RC	R2	Rres	RR	
ALU(2)	←	T2	2	RCY	R4	RCX	1	4	T1
MUL(1)	←	2	RD	T1	T2				
MUL(2)	←	RR	RC	R3	R1	T1			
DIV(1)	←	4							
DIV(2)	←	MEM(DO)							
ALU_INT(1)	←	Radr	RI						
ALU_INT(2)	←	CInt 1							

Tabelle 1: Datentransferoperationen der Datenpfadarchitektur. Bei Konstanten wurde auf das Präfix *Const* verzichtet.

Anschließend wurde die Datenpfadarchitektur erstellt. Als Bild genügt hier der Ausdruck aus Cadence.

3.4. Registertransferfolgen

In Abbildung 1 seht ihr ein Beispiel der Registertransferfolgen, als formschöne Vektorgrafik. Erstellt in drawio. Für den Export wurden die Einstellungen in Abbildung 2 verwendet.

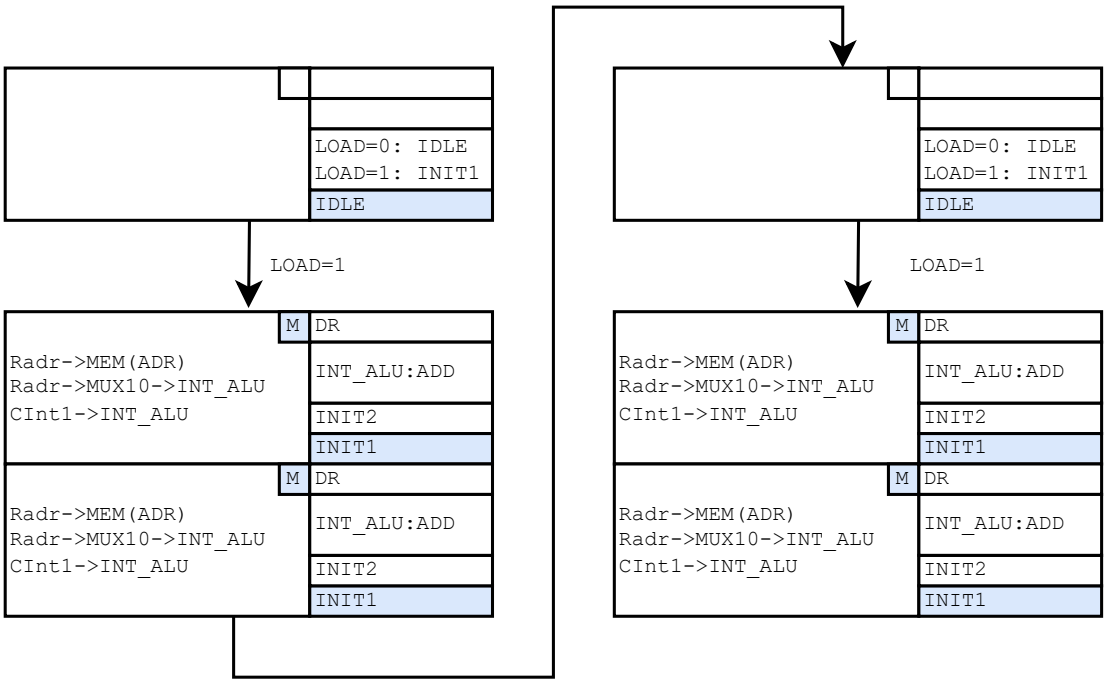


Abbildung 1: Registertransferfolgen. Erstellt mit drawio. Source befindet sich in dieser zip Datei.

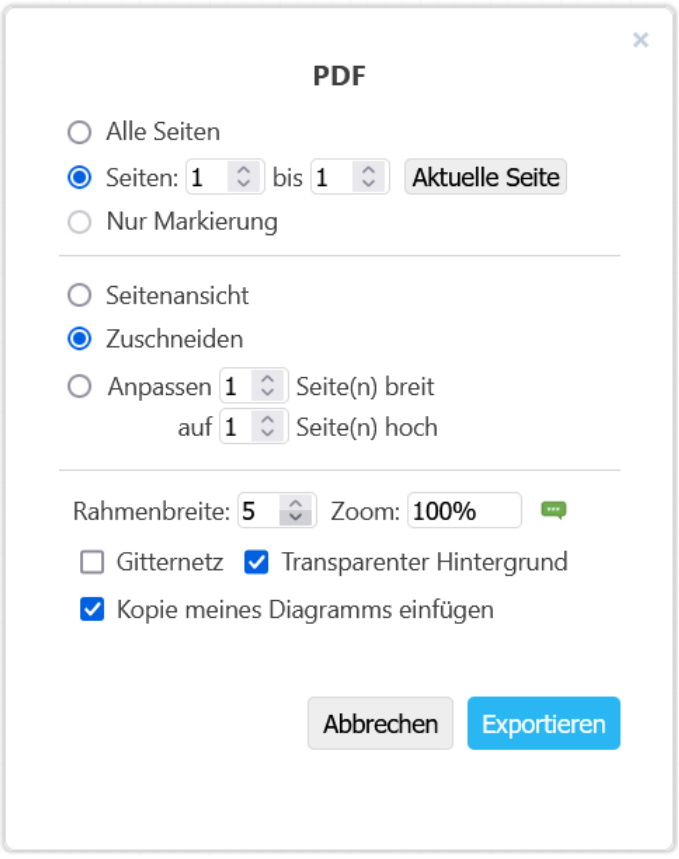


Abbildung 2: Exporteinstellungen drawio.

3.5. Zustandsgraph

Nicht vergessen: Die Zustandskodierung, wie in Tabelle 2 dargestellt.

Zustand	Hex-Code	Zustand	Hex-Code
IDLE	00	CLOOP2	0E
INIT1	01	CLOOP3	0F
INIT2	02	ILOOP1	10
INIT3	03	ILOOP2	11
INIT4	04	ILOOP3	12
INIT5	05	ILOOP4	13
INIT6	06	ILOOP5	14
INIT7	07	ILOOP6	15
INIT8	08	ILOOP7	16
INIT9	09	CLOOP4	17
RLOOP1	0A	CLOOP5	18
RLOOP2	0B	CLOOP6	19
RLOOP3	0C	RLOOP4	1A
CLOOP1	0D	RLOOP5	1B
		RLOOP6	1C

Tabelle 2: Zustandskodierung

3.6. Gesamtschaltung

4. Simulation

In den folgenden Unterkapiteln wurde die Gesamtschaltung mithilfe der Verilog Testbench simuliert. Dafür müssen die beiden Simulationsparameter *resolution* und *iteration count* im Speicher liegen. Diese werden in eine Textdatei geschrieben, welche anschließend von der Testbench eingelesen werden. Es wurden folgende Simulationsparameter verwendet:

Parameter	Dezimal	Hexadezimal	Adresse im Speicher
resolution	25	0x00000019	0x00000000
iteration count	50	0x00000032	0x00000001

4.1. Verifikation

Wie habt ihr Sichergestellt, dass euer Algorithmus tut, was er tut?

4.2. Schaltungssynthese

4.3. Layoutsynthese

5. Zusammenfassung

Die Simulation des Schaltkreises hat gezeigt, dass er wie erwartet funktioniert. Ihr solltet noch ein paar Worte sagen zu:

- Abweichungen zur Referenzimplementierung
- Trade-off Geschwindigkeit - Area
- Verbesserungsmöglichkeiten

Eine halbe Seite genügt.

Appendix

A. Quellcode

A.1. Referenzimplementierung

```
#include<stdio.h>

int main() {
  printf("Hello World\n");
  return 0;
}
```

A.2. FSM

A.3. Controllogic

B. Reports

B.1. Area

B.2. Power

B.3. Timing